

coefs_v3

December 27, 2023

```
[ ]: import numpy as np
      from sympy import symbols, expand, binomial
      import re
```

En esta nueva versión los coeficientes de g_k se calculan de forma directa y no numérica.

```
[ ]: ROUND_VALUE = 10
```

```
[ ]: # Función auxiliar para poder tratar el output de la
      # función expand
      def transform_key(key):
          key = str(key)

          if key == '1':
              return 0

          if key == 'w':
              return 1

          match = re.search(r'w\*\*([0-9]+)', key)
          return int(match.group(1))
```

```
[ ]: w = symbols('w')
      def calculate_coefficients_of_gk(k):
          '''
          Devuelve coeficientes de la serie de cosenos de  $g_k$ 
          '''
          g_k = 0

          for l in range(k + 1):
              g_k += binomial(2 * k + 1, k - l) * (1 - w)**(k - l) * (1 + w)**l

          g_k *= (1 + w)**(k + 1) / (2**(2 * k + 1))

          expanded_gk = expand(g_k).as_coefficients_dict()
```

```

coeff_dict = {transform_key(k): v for k, v in
               expanded_gk.items()}

return coeff_dict

```

```

[ ]: def get_conversion_dict(max_exponent):
    """
        Devuelve un diccionario exponente, lista de exponentes
        con la equivalencia  $\cos(w)^n = \sum \text{from } -n \text{ to } n e^{ikw}$ 
    """

    #  $\cos(w)^0 = 1 = e^0$ ,  $\cos(w) = 1/2 (e^{ikw} + e^{-ikw})$ 
    conversion_dict = {0: [1], 1: [1/2, 0, 1/2]}

    coef_cos = [1/2, 0, 1/2]

    for k in range(2, max_exponent+1):
        coef_cos = np.convolve(coef_cos, [1/2, 0, 1/2])
        conversion_dict[k] = coef_cos

    return conversion_dict

def transform_cos_to_exp(coeff_dict, conversion_dict=None):
    """
        Transforma los coeficientes de la serie de cosenos de a una serie de
        exponenciales
    """

    # Si llamamos esta función varias veces podemos precomputar
    # el diccionario de conversión
    if not conversion_dict:
        max_exponent = max(coeff_dict.keys())
        conversion_dict = get_conversion_dict(max_exponent)

    exponential_coefs = [0]*(2*max_exponent+1)

    for l, c in coeff_dict.items():
        j = max_exponent - l
        coefs = [0]*j + [c*v for v in conversion_dict[l]] + [0]*j
        exponential_coefs = [sum(x) for x in zip(exponential_coefs, coefs)]

```

```
return exponential_coefs
```

```
[ ]: def get_coefficients_gk(k):
    '''
    Calcula los coeficientes de la función g_k
    cuando se escribe como serie de exp(ikw)
    '''

    cos_coefficients = calculate_coefficients_of_gk(k)
    exp_coefficients = transform_cos_to_exp(cos_coefficients)
    return exp_coefficients
```

A partir de aquí, no hay ningún cambio con respecto a la versión 1 salvo que las raíces se calculan de forma SIMBÓLICA.

```
[ ]: from sympy import Poly
from sympy.abc import x

def get_roots_inside_unit_circle(coefs):
    '''
    Calcula las raíces dentro del círculo unitario
    '''

    p = Poly(coefs,x)
    _roots = p.all_roots() #Cálculo simbólico

    _roots = sorted(_roots, key=abs)
    return _roots[:len(_roots)//2]
```

Hallamos las raíces de $P(z)$ y nos quedamos con las que están dentro del círculo unidad. Con estas raíces construimos el filtro m . Los coeficientes de $m(\xi)$ los calculamos a partir de sus raíces.

```
[ ]: def get_filter_coefficients(roots):
    '''
    Calcula los coeficientes del filtro a partir de las raíces
     $h_0 = (-1)^n * r_1 * r_2 * \dots * r_n$ 
    ....
     $h_{(n-1)} = -(r_1 + r_2 + \dots + r_n)$ 
     $h_n = 1$ 
    '''

    coefs = [1]
    for r in roots:
        coefs = np.convolve(coefs, [1, -r])

    f = np.sqrt(2)/sum(coefs)
```

```
#Normalizar
return [f*c for c in coefs]
```

```
[ ]: import sympy
def get_Daubechies_filter(k):
    coefs = get_coefficients_gk(k)
    roots = get_roots_inside_unit_circle(coefs)
    filter_coefs = get_filter_coefficients(roots)

    # Los coeficientes han de ser reales. Forzamos la parte imaginaria a 0

    filter_coefs = [sympy.re(c.evalf()) for c in filter_coefs]

    return filter_coefs
```

```
[ ]: # imprimimos valores de los filtros.
# (k+1 para seguir con la notación de la tabla que aparece en Daubechies Ten-
↳ Lectures On Wavelets)
for k in range(1, 10):
    print(k+1, get_Daubechies_filter(k))
```

```
2 [0.482962913144534, 0.836516303737808, 0.224143868042013, -0.129409522551260]
3 [0.332670552950083, 0.806891509311093, 0.459877502118492, -0.135011020010255,
-0.0854412738820267, 0.0352262918857095]
4 [0.230377813308897, 0.714846570552916, 0.630880767929859, -0.0279837694168599,
-0.187034811719093, 0.0308413818355608, 0.0328830116668852, -0.0105974017850690]
5 [0.160102397974193, 0.603829269797190, 0.724308528437773, 0.138428145901321,
-0.242294887066382, -0.0322448695846384, 0.0775714938400457,
-0.00624149021279827, -0.0125807519990820, 0.00333572528547377]
6 [0.111540743350109, 0.494623890398453, 0.751133908021095, 0.315250351709198,
-0.226264693965440, -0.129766867567262, 0.0975016055873231, 0.0275228655303057,
-0.0315820393174860, 0.000553842201161496, 0.00477725751094551,
-0.00107730108530848]
7 [0.0778520540850092, 0.396539319481917, 0.729132090846235, 0.469782287405193,
-0.143906003928565, -0.224036184993875, 0.0713092192668303, 0.0806126091510831,
-0.0380299369350144, -0.0165745416306669, 0.0125509985560998,
0.000429577972921367, -0.00180164070404749, 0.000353713799974520]
8 [0.0544158422431040, 0.312871590914300, 0.675630736297290, 0.585354683654207,
-0.0158291052563493, -0.284015542961547, 0.000472484573913283,
0.128747426620478, -0.0173693010018075, -0.0440882539307948, 0.0139810279173983,
0.00874609404740578, -0.00487035299345158, -0.000391740373376947,
0.000675449406450569, -0.000117476784124770]
```

KeyError

Traceback (most recent call last)

File ~/.local/lib/python3.10/site-packages/sympy/core/evalf.py:1481, in evalf(x, prec, options)

```

1480 try:
-> 1481     rf = evalf_table[type(x)]
1482     r = rf(x, prec, options)

```

KeyError: <class 'sympy.polys.rootoftools.ComplexRootOf'>

During handling of the above exception, another exception occurred:

KeyboardInterrupt Traceback (most recent call last)

Cell In[88], line 4

```

1 # imprimimos valores de los filtros.
2 # (k+1 para seguir con la notación de la tabla que aparece en Daubechies
↳Ten Lectures On Wavelets)
3 for k in range(1, 10):
----> 4     print(k+1, get_Daubechies_filter(k))

```

Cell In[86], line 10, in get_Daubechies_filter(k)

```

5 filter_coefs = get_filter_coefficients(roots)
7 # Los coeficientes han de ser reales. Forzamos la parte imaginaria a 0
8 # para contrarestar errores de redondeo
---> 10 filter_coefs = [sympy.re(c.evalf()) for c in filter_coefs]
12 return filter_coefs

```

Cell In[86], line 10, in <listcomp>(.0)

```

5 filter_coefs = get_filter_coefficients(roots)
7 # Los coeficientes han de ser reales. Forzamos la parte imaginaria a 0
8 # para contrarestar errores de redondeo
---> 10 filter_coefs = [sympy.re(c.evalf()) for c in filter_coefs]
12 return filter_coefs

```

File ~/.local/lib/python3.10/site-packages/sympy/core/evalf.py:1647, in

```

↳EvalfMixin.evalf(self, n, subs, maxn, chop, strict, quad, verbose)
1645     options['quad'] = quad
1646 try:
-> 1647     result = evalf(self, prec + 4, options)
1648 except NotImplementedError:
1649     # Fall back to the ordinary evalf
1650     if hasattr(self, 'subs') and subs is not None: # issue 20291

```

File ~/.local/lib/python3.10/site-packages/sympy/core/evalf.py:1482, in evalf(x

```

↳prec, options)
1480 try:
1481     rf = evalf_table[type(x)]
-> 1482     r = rf(x, prec, options)
1483 except KeyError:
1484     # Fall back to ordinary evalf if possible
1485     if 'subs' in options:

```

```

File ~/.local/lib/python3.10/site-packages/sympy/core/evalf.py:648, in
↳evalf_mul(v, prec, options)
    646 from .numbers import Float
    647 for arg in args:
--> 648     result = evalf(arg, prec, options)
    649     if result is S.ComplexInfinity:
    650         special.append(result)

File ~/.local/lib/python3.10/site-packages/sympy/core/evalf.py:1482, in evalf(x
↳prec, options)
    1480 try:
    1481     rf = evalf_table[type(x)]
-> 1482     r = rf(x, prec, options)
    1483 except KeyError:
    1484     # Fall back to ordinary evalf if possible
    1485     if 'subs' in options:

File ~/.local/lib/python3.10/site-packages/sympy/core/evalf.py:776, in
↳evalf_pow(v, prec, options)
    773 # Exponentiation by p magnifies relative error by |p|, so the
    774 # base must be evaluated with increased precision if p is large
    775 prec += int(math.log(abs(p), 2))
--> 776 result = evalf(base, prec + 5, options)
    777 if result is S.ComplexInfinity:
    778     if p < 0:

File ~/.local/lib/python3.10/site-packages/sympy/core/evalf.py:1482, in evalf(x
↳prec, options)
    1480 try:
    1481     rf = evalf_table[type(x)]
-> 1482     r = rf(x, prec, options)
    1483 except KeyError:
    1484     # Fall back to ordinary evalf if possible
    1485     if 'subs' in options:

File ~/.local/lib/python3.10/site-packages/sympy/core/evalf.py:600, in
↳evalf_add(v, prec, options)
    597 while 1:
    598     options['maxprec'] = min(oldmaxprec, 2*prec)
--> 600     terms = [evalf(arg, prec + 10, options) for arg in v.args]
    601     n = terms.count(S.ComplexInfinity)
    602     if n >= 2:

File ~/.local/lib/python3.10/site-packages/sympy/core/evalf.py:600, in
↳<listcomp>(.0)
    597 while 1:
    598     options['maxprec'] = min(oldmaxprec, 2*prec)
--> 600     terms = [evalf(arg, prec + 10, options) for arg in v.args]

```

```

601     n = terms.count(S.ComplexInfinity)
602     if n >= 2:

File ~/.local/lib/python3.10/site-packages/sympy/core/evalf.py:1482, in evalf(x, prec, options)
    1480 try:
    1481     rf = evalf_table[type(x)]
-> 1482     r = rf(x, prec, options)
    1483 except KeyError:
    1484     # Fall back to ordinary evalf if possible
    1485     if 'subs' in options:

File ~/.local/lib/python3.10/site-packages/sympy/core/evalf.py:696, in evalf_mul(v, prec, options)
    694 elif i == last and arg is S.One:
    695     continue
--> 696 re, im, re_acc, im_acc = evalf(arg, working_prec, options)
    697 if re and im:
    698     complex_factors.append((re, im, re_acc, im_acc))

File ~/.local/lib/python3.10/site-packages/sympy/core/evalf.py:1487, in evalf(x, prec, options)
    1485 if 'subs' in options:
    1486     x = x.subs(evalf_subs(prec, options['subs']))
-> 1487 xe = x._eval_evalf(prec)
    1488 if xe is None:
    1489     raise NotImplementedError

File ~/.local/lib/python3.10/site-packages/sympy/polys/rootoftools.py:919, in ComplexRootOf._eval_evalf(self, prec, **kwargs)
    917 """Evaluate this complex root to the given precision."""
    918 # all kwargs are ignored
--> 919 return self.eval_rational(n=prec_to_dps(prec))._evalf(prec)

File ~/.local/lib/python3.10/site-packages/sympy/polys/rootoftools.py:990, in ComplexRootOf.eval_rational(self, dx, dy, n)
    988 dx = abs(interval.center[0]*rtol)
    989 dy = abs(interval.center[1]*rtol)
--> 990 interval = interval.refine_size(dx, dy)
    991 c = interval.center
    992 real, imag = map(Rational, c)

File ~/.local/lib/python3.10/site-packages/sympy/polys/rootisolation.py:2183, in ComplexInterval.refine_size(self, dx, dy)
    2181 expr = self
    2182 while not (expr.dx < dx and expr.dy < dy):
-> 2183     expr = expr._inner_refine()
    2185 return expr

```

```
File ~/.local/lib/python3.10/site-packages/sympy/polys/rootisolation.py:2153, in
↳ ComplexInterval._inner_refine(self)
```

```
    2150 dom = self.dom
    2152 if s - u > t - v:
-> 2153     D_L, D_R =
↳ _vertical_bisection(1, (u, v), (s, t), I, Q, F1, F2, f1, f2, dom)
    2155     if D_L[0] == 1:
    2156         _, a, b, I, Q, F1, F2 = D_L
```

```
File ~/.local/lib/python3.10/site-packages/sympy/polys/rootisolation.py:1366, in
↳ _vertical_bisection(N, a, b, I, Q, F1, F2, f1, f2, F)
```

```
    1363 f1V = dmp_eval_in(f1, x, 0, 1, F)
    1364 f2V = dmp_eval_in(f2, x, 0, 1, F)
-> 1366 I_V =
↳ dup_isolate_real_roots_list([f1V, f2V], F, inf=v, sup=t, fast=True, strict=True, basis=True)
    1368 I_L1_L, I_L1_R = [], []
    1369 I_L2_L, I_L2_R = I_V, I_L2
```

```
File ~/.local/lib/python3.10/site-packages/sympy/polys/rootisolation.py:667, in
↳ dup_isolate_real_roots_list(polys, K, eps, inf, sup, strict, basis, fast)
```

```
    664 if zeros and j > 0:
    665     zero_indices[i] = j
-> 667 for f, k in dup_factor_list(p, K)[1]:
    668     f = tuple(f)
    670     if f not in factors_dict:
```

```
File ~/.local/lib/python3.10/site-packages/sympy/polys/factortools.py:1342, in
↳ dup_factor_list(f, K0)
```

```
    1339 K = K0
    1341 if K.is_ZZ:
-> 1342     coeff, factors = dup_zz_factor(f, K)
    1343 elif K.is_Poly:
    1344     f, u = dmp_inject(f, 0, K)
```

```
File ~/.local/lib/python3.10/site-packages/sympy/polys/factortools.py:688, in
↳ dup_zz_factor(f, K)
```

```
    685 H = dup_zz_cyclotomic_factor(g, K)
    687 if H is None:
-> 688     H = dup_zz_zassenhaus(g, K)
    690 factors = dup_trial_division(f, H, K)
    691 return cont, factors
```

```
File ~/.local/lib/python3.10/site-packages/sympy/polys/factortools.py:377, in
↳ dup_zz_zassenhaus(f, K)
```

```
    375 G = [b]
    376 for i in S:
-> 377     G = dup_mul(G, g[i], K)
```

```
378 G = dup_trunc(G, pl, K)
379 G = dup_primitive(G, K)[1]
```

File ~/.local/lib/python3.10/site-packages/sympy/polys/densearith.py:767, in

```
↳dup_mul(f, g, K)
    764     coeff = K.zero
    766     for j in range(max(0, i - dg), min(df, i) + 1):
--> 767         coeff += f[j]*g[i - j]
    769     h.append(coeff)
    771 return dup_strip(h)
```

KeyboardInterrupt: